

- **Ciclo Formativo:** DAM
- **Módulo:** Programación de Procesos y Servicios
- **Unidad de trabajo:** 01.
- **Tarea** 01.

Enunciado

Ejercicio 1 (4 Puntos)

Este ejercicio consta de 3 partes.

- A) Implementa una aplicación que ordena en sentido de mayor a menor un número indeterminado de números que se recibirá a través de su entrada estándar. Para indicar que hemos finalizado de introducir números, se introducirá un cero. La aplicación se llamará ordenarNumeros. Para llamar al programa ejecutaremos el siguiente comando:
java -jar ordenarNumeros.jar

```
D:>java -jar ordenarNumeros.jar
52      // Números introducidos.
36
41
95
65
2
0      // Finalizamos introducir números y pasamos el mostrarlos de forma
95      // ordenada
65
52
41
36
2
D:>
```

- B) Implementa una aplicación, llamada 'aleatorios', que genere un numero X de números aleatorios (entre 0 y 100), el número de números a generar de forma aleatoria, se pasará a través de la llamada al programa, y que los escriba en su salida estándar. Para llamar al programa ejecutaremos el siguiente comando: java -jar aleatorios.jar 20. Generará 20 números de forma aleatoria:

```
D:\> java -jar aleatorios.jar 10
19
50
52
84
35
85
88
74
```



```
81
72
D:\>
```

- C) Realiza un pequeño manual (tipo "¿Cómo se hace?" o "HowTo"), utilizando un editor de textos (tipo word o writer) en el que indiques, con pequeñas explicaciones y capturas, cómo has probado la ejecución de las aplicaciones que has implementado en este ejercicio. Entre las pruebas que hayas realizado, debes incluir una prueba en la que utilizando el operador "|" (tubería) redirijas la salida de la aplicación 'aleatorios' a la entrada de la aplicación 'ordenarNumeros'. De la siguiente forma:

```
D:\>java -jar aleatorios.jar 10 | java -jar ordenarNumeros.jar
87
82
71
60
57
57
44
30
25
8
D:\>
```

Ejercicio 2 (6 puntos)

Este ejercicio consta de 3 partes.

- A) Implementa una aplicación que escriba en un fichero indicado por el usuario conjuntos de letras generadas de forma aleatoria (sin sentido real). Escribiendo cada conjunto de letras en una línea distinta. El número de conjuntos de letras a generar por el proceso, también será dado por el usuario en el momento de su ejecución. Esta aplicación se llamará "**lenguaje**" y como ejemplo, podrá ser invocada así:

```
D:\>java -jar lenguaje.jar 43 prueba.txt
```

Genera el fichero prueba.txt con 43 palabras.

- B) Implementa una aplicación, llamada '**colaborar**', que lance al menos 10 instancias de la aplicación "**lenguaje**". Haciendo que todas ellas, colaboren en generar un gran fichero de palabras. Cada instancia generará un número creciente de palabras de 10, 20, 30, ... Por supuesto, cada proceso seguirá escribiendo su palabra en una línea independiente de las otras. Es decir, si lanzamos 10 instancias de "**lenguaje**", al final, debemos tener en el fichero $10 + 20 + 30 + \dots + 100 = 550$ líneas.

```
d:\>java -jar colaborar.jar 10 datos.txt
```

- C) Realiza un pequeño manual (tipo "¿Cómo se hace?" o "HowTo"), utilizando un editor de textos (tipo word o writer) en el que indiques, con pequeñas explicaciones y capturas, cómo has probado la ejecución de las aplicaciones que has implementado en este ejercicio.
-

Ejercicio 1) 4 puntos, de los cuales:

- Aplicación "**ordenarNumeros**" aceptando un número indeterminado de valores → (1,5 puntos).
- Aplicación "**aleatorios**" → (0,75 puntos.)
- Documentación del código y Javadoc en ambas aplicaciones → (0,75 puntos).
- Mini manual de uso y pruebas → (1 punto).

Ejercicio 2) 6 puntos, de los cuales:

- Aplicación "**lenguaje**" (correcta para su ejecución en modo aislado o secuencial) → (1, 5 puntos).
- Aplicación "**colaborar**" (lanzando aplicaciones simultáneas) → (1 puntos).
- Aplicación "**lenguaje**" (correcta para su ejecución en un entorno concurrente de ejecución compartiendo un recurso) → (1,75 puntos)
- Mini manual de uso y pruebas → (1 puntos).
- Documentación del código y Javadoc en ambas aplicaciones → (0,75 puntos).

Se tendrá en cuenta que:

La ejecución de los programas produce el resultado esperado.
No se produce interbloqueo ni inanición

Recursos necesarios para realizar la Tarea.

- IDE NetBeans.
- Contenidos de la unidad.
- Ejemplos expuestos en el contenido de la unidad.

Indicaciones de entrega.

Los documentos deben tener tamaño de página A4, estilo de letra Times New Roman, tamaño 12 e interlineado normal, si se introduce el código debe emplearse una letra monoespacia, (courier new).

Debes crear una carpeta para el ejercicio, y en ella una carpeta para cada actividad. En cada carpeta incluye los proyectos y documentos correspondientes. Comprímelo todo en un fichero.

Por ejemplo, tendremos: PSP01_Tarea con los directorios, Ejercicio1 y Ejercicio2; en Ejercicio1 tendremos dos proyectos y un documento; en Ejercicio2, dos proyectos y un documento.

Una vez realizada la tarea elaborarás un único documento donde figuren las respuestas correspondientes. El envío se realizará a través de la plataforma de la forma establecida para ello, y el archivo se nombrará siguiendo las siguientes pautas:

apellido1_apellido2_nombre_SIGxx_Tarea

Asegúrate que el nombre no contenga la letra ñ, tildes ni caracteres especiales extraños. Así por ejemplo la alumna Begoña Sánchez Mañas para la primera unidad del MP de PSP, debería nombrar esta tarea como...

sanchez_manas_begona_PSP01_Tarea

En el caso de que algún problema de compilación la calificación será de cero puntos. El entorno de programación usado ha sido netbeans, 22 o posterior.

Los proyectos en Java son del tipo:
Java with Ant → Java Application.

